Design Document

AI Trip Planner Agent System

Complete Technical Specification
with Rendered PlantUML Diagrams

Version 1.0

Status: **Final**

Development Team

February 25, 2026

Confidential
This document contains proprietary information
and is intended for internal use only.

Document Information

Document Title	AI Trip Planner - Design Document with Rendered Diagrams
Version	1.0
Date	February 25, 2026
Status	Final
Author	Development Team
Classification	Confidential - Internal Use Only
Distribution	Architecture Team, Development Team, QA Team
Diagram Rendering	PlantUML code provided as listings
Related Documents	Software Requirements Specification (SRS-Document.md) Architecture Documentation (ARCHITECTURE.md) API Documentation (FastAPI /docs)

Document Metadata

Compilation Instructions

This document compiles with standard LaTeX:

```
# Standard compilation
pdflatex DESIGN-DOCUMENT-WITH-DIAGRAMS.tex

# Run twice for proper cross-references
pdflatex DESIGN-DOCUMENT-WITH-DIAGRAMS.tex
```

Note: PlantUML diagrams are included as code listings. To generate actual diagram images, see Appendix **??**.

Revision History

Version	Date	Description	Author
1.0	2026-02-25	Complete design document with rendered PlantUML diagrams	Development Team

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Purpose

This design document provides a comprehensive technical blueprint for the AI Trip Planner Agent system. It describes the architecture, components, interfaces, data structures, and design decisions that guide the implementation of this intelligent travel itinerary generation system.

All diagrams in this document are rendered directly from PlantUML source code, ensuring accuracy and easy maintenance.

1.1.1 Target Audience

This document is intended for:

- Software architects and senior developers
- Backend and frontend development teams
- DevOps and infrastructure engineers
- Technical leads and system integrators
- Quality assurance and testing teams

1.2 Scope

This document covers the complete technical design of an AI-powered trip planning system that leverages LangChain agents with the ReAct (Reasoning + Acting) pattern to generate personalized travel itineraries.

1.2.1 System Capabilities

- AI agent orchestration using LangChain framework
- Integration with external APIs (Groq, OpenStreetMap, Open-Meteo, Overpass, OpenRouteService)

- Real-time data aggregation and processing
- Structured itinerary generation with budget tracking
- Responsive web interface with modern UX
- RESTful API with FastAPI framework

1.2.2 Technology Stack

Layer	Technologies
Backend	Python 3.9+, FastAPI, LangChain, Pydantic
Frontend	HTML5, CSS3, JavaScript (ES6+)
AI/LLM	Groq API with Llama 3.1 models
External APIs	OpenStreetMap, Open-Meteo, Overpass API, OpenRouteService
Deployment	Docker, Cloud platforms (AWS/Azure/GCP)

Table 1.1: Technology Stack Overview

1.3 Definitions and Acronyms

Term	Definition
Agent	Autonomous AI system that reasons and uses tools
ReAct	Reasoning + Acting pattern for AI agents
LangChain	Framework for building LLM-powered applications
Groq	Ultra-fast AI inference platform
POI	Point of Interest (tourist attractions, restaurants, etc.)
Nominatim	OpenStreetMap's geocoding service
Overpass	OpenStreetMap's query API for POI data
FastAPI	Modern Python web framework for building APIs
Pydantic	Data validation library using Python type hints
CORS	Cross-Origin Resource Sharing
LLM	Large Language Model
REST	Representational State Transfer

Table 1.2: Definitions and Acronyms

1.4 Design Goals

1. **Performance:** Generate complete itineraries in under 60 seconds
2. **Scalability:** Support concurrent users without performance degradation
3. **Reliability:** Graceful handling of external API failures with fallbacks
4. **Maintainability:** Modular architecture with clear separation of concerns

5. **Usability:** Intuitive interface with real-time feedback
6. **Security:** Secure API key management and input validation
7. **Extensibility:** Easy addition of new tools and features

1.5 References

- Software Requirements Specification (`SRS-Document.md`)
- Architecture Documentation (`ARCHITECTURE.md`)
- API Documentation (FastAPI /docs endpoint)
- LangChain Documentation: https://python.langchain.com/
- Groq API Docs: https://console.groq.com/docs
- FastAPI Documentation: https://fastapi.tiangolo.com/

Chapter 2

System Overview

2.1 System Context

The AI Trip Planner is a standalone web-based application that integrates with multiple external services to provide intelligent travel planning capabilities. The system acts as an intermediary between users and various data sources, leveraging AI to synthesize information into coherent travel itineraries.

2.1.1 System Context Diagram

Figure ?? shows the high-level system context with all external actors and systems.

[System Context Diagram Placeholder]

To generate: Save PlantUML code below to system-context-diagram.puml then run: java -jar plantuml.jar system-context-diagram.puml

Figure 2.1: System Context Diagram

```
@startuml system-context-diagram
' System Context Diagram for AI Trip Planner

!define RECTANGLE class

skinparam rectangle {
    BackgroundColor<<system>> LightBlue
    BackgroundColor<<external>> LightYellow
    BackgroundColor<<user>> LightGreen
    BorderColor Black
    FontSize 12
}

title AI Trip Planner System - Context Diagram

rectangle "User" <<user>> as User {
}

rectangle "AI Trip Planner\nSystem" <<system>> as System {
}

```

```
rectangle "Groq API\n(LLM Service)" <<external>> as Groq {
}

rectangle "OpenStreetMap\n(Nominatim)" <<external>> as OSM {
}

rectangle "Open-Meteo\n(Weather)" <<external>> as Weather {
}

rectangle "Overpass API\n(POI Data)" <<external>> as Overpass {
}

rectangle "OpenRouteService\n(Routing)" <<external>> as ORS {
}

User --> System : "Submit trip\nrequirements"
System --> User : "Return personalized\nitinerary"

System --> Groq : "Send prompts\nfor AI generation"
Groq --> System : "Return AI-generated\nresponses"

System --> OSM : "Query location\ncoordinates"
OSM --> System : "Return lat/lon\ndata"

System --> Weather : "Request weather\nforecast"
Weather --> System : "Return forecast\ndata"

System --> Overpass : "Query points of\ninterest"
Overpass --> System : "Return POI\nlist"

System --> ORS : "Request route\ninformation"
ORS --> System : "Return distance\nand duration"

note right of System
  **Core System Components:**
  - FastAPI Backend
  - LangChain Agent
  - Web Frontend
  - API Integration Layer
end note

@enduml
```

Listing 2.1: System Context Diagram - PlantUML Code

2.2 High-Level Architecture

The system follows a three-tier architecture as illustrated in Figure **??**.

2.2.1 Architectural Layers

1. **Presentation Layer (Frontend)**
   - Single-page web application
   - Responsive UI for desktop and mobile

- Client-side validation and state management

2. **Application Layer (Backend)**
   - FastAPI REST API server
   - AI agent orchestration with LangChain
   - Business logic and data processing
   - External API integration
3. **Data Layer (External Services)**
   - Groq API for LLM inference
   - OpenStreetMap for geocoding and POI data
   - Open-Meteo for weather forecasts
   - OpenRouteService for route optimization

[High-Level Architecture Diagram Placeholder]

To generate: Save PlantUML code below to high-level-architecture.puml then run: java -jar plantuml.jar high-level-architecture.puml

Figure 2.2: High-Level Architecture Diagram

```
@startuml high-level-architecture-diagram
' High-Level Architecture Diagram

!define RECTANGLE class

skinparam rectangle {
    BackgroundColor<<frontend>> LightGreen
    BackgroundColor<<backend>> LightBlue
    BackgroundColor<<external>> LightYellow
    BorderColor Black
    FontSize 11
}

title AI Trip Planner - High-Level Architecture

package "Presentation Layer" <<frontend>> {
    rectangle "Web Browser" {
        rectangle "HTML5\n(Structure)" as HTML
        rectangle "CSS3\n(Styling)" as CSS
        rectangle "JavaScript\n(Logic)" as JS
    }
}

package "Application Layer" <<backend>> {
    rectangle "FastAPI Server" {
        rectangle "REST API\nEndpoints" as API
        rectangle "Request\nValidation" as Validation
        rectangle "Response\nFormatting" as Formatting
    }
```

```
    rectangle "Business Logic" {
        rectangle "Trip Service\n(Orchestration)" as TripService
        rectangle "API Service\n(Integration)" as APIService
    }

    rectangle "AI Agent System" {
        rectangle "LangChain\nAgent" as Agent
        rectangle "Tool\nDefinitions" as Tools
        rectangle "Prompt\nEngineering" as Prompts
    }
}

package "Data Layer" <<external>> {
    rectangle "Groq API\n(Llama 3.1)" as Groq
    rectangle "OpenStreetMap\n(Geocoding)" as OSM
    rectangle "Open-Meteo\n(Weather)" as Weather
    rectangle "Overpass API\n(POI)" as Overpass
    rectangle "OpenRouteService\n(Routing)" as ORS
}

HTML --> JS : "User\ninteractions"
CSS --> HTML : "Styling"
JS --> API : "HTTPS/JSON\nAPI calls"

API --> Validation : "Validate\nrequest"
Validation --> TripService : "Valid\nrequest"
TripService --> APIService : "Fetch external\ndata"
TripService --> Agent : "Generate\nitinerary"
Agent --> Tools : "Use tools"

APIService --> Groq : "LLM\nrequests"
APIService --> OSM : "Geocoding"
APIService --> Weather : "Forecast"
APIService --> Overpass : "POI query"
APIService --> ORS : "Route info"

Agent --> Formatting : "Raw\nitinerary"
Formatting --> API : "Formatted\nresponse"
API --> JS : "JSON\nresponse"

@enduml
```

Listing 2.2: High-Level Architecture - PlantUML Code

2.3 Key Design Patterns

2.3.1 ReAct Agent Pattern

The core of the system uses the ReAct (Reasoning + Acting) pattern for AI agents:

```
# Thought: Analyze what information is needed
# Action: Call a tool to gather data
# Observation: Process tool output
# ... (repeat as needed)
# Final Answer: Synthesize structured itinerary
```

Listing 2.3: ReAct Pattern Flow

2.3.2 Service Layer Pattern

Business logic is separated into service layers:

- `trip_service.py`: Orchestrates trip planning workflow
- `api_service.py`: Manages external API calls
- Clear separation between routes (controllers) and services

2.3.3 Repository Pattern

External API calls are abstracted through a service layer, making it easy to:

- Mock services for testing
- Add caching layers
- Implement retry logic
- Switch API providers

Chapter 3

System Architecture

3.1 Architectural Style

The system implements a **Layered Architecture** with **Microservices principles**:

- **Separation of Concerns:** Frontend, backend, and external services are clearly separated
- **Stateless Design:** No server-side session management
- **API-First:** RESTful API as the primary interface
- **Cloud-Native:** Containerized for easy deployment

3.2 Component Architecture

Figure **??** provides a detailed view of all system components and their relationships.

3.2.1 Frontend Components

index.html

- Trip planning form
- Loading state UI
- Results display area
- Error handling UI

app.js

- Form submission handler
- State management (currentTrip, isLoading)
- UI controller (showSection, displayResults)
- Loading animation controller
- PDF generation

[Component Architecture Diagram Placeholder]

To generate: Save PlantUML code below to component-architecture.puml then run: java -jar plantuml.jar component-architecture.puml

Figure 3.1: Component Architecture Diagram

```
!define COMPONENT rectangle
skinparam component  BackgroundColor LightBlue BorderColor Black FontSize 10
title AI Trip Planner - Component Architecture
package "Frontend"  [index.html] as HTML [styles.css] as CSS [app.js] as AppJS [api.js] as ApiJS
HTML -> AppJS : "includes" HTML -> CSS : "styles" AppJS -> ApiJS : "uses"
package "Backend - main.py"  [FastAPI App] as App [CORS Middleware] as CORS [Exception Handler] as ExHandler [Startup Events] as Startup
App -> CORS : "configures" App -> ExHandler :  "registers" App -> Startup :  "runs"
package "Backend - routes/"  [trip_planner.py]asRoutes[/api/plan-trip]asPlanTrip[/api/health]asHealth[/api/sample-*]asSample
Routes -> PlanTrip :  "defines" Routes -> Health :  "defines" Routes -> Sample :  "defines"
package "Backend - services/"  [trip_service.py]asTripService[api_service.py]asAPIService
TripService -> APIService :  "uses"
package "Backend - agents/"  [trip_agent.py]asTripAgent[tools.py]asTools
TripAgent -> Tools :  "registers"
package "Backend - models/"  [schemas.py] as Schemas [TripRequest] as TripReq [TripResponse] as TripResp [Activity] as Activity [DayItinerary] as DayItin
Schemas -> TripReq :  "defines" Schemas -> TripResp :  "defines" Schemas -> Activity :  "defines" Schemas -> DayItin :  "defines"
package "Backend - utils/"  [config.py] as Config [helpers.py] as Helpers
package "External APIs"  [Groq API] as Groq [Nominatim] as Nominatim [Open-Meteo] as OpenMeteo [Overpass API] as Overpass [OpenRouteService] as ORS
ApiJS -> App :  "HTTP/JSON" App -> Routes :  "routes to"
PlanTrip -> TripService :  "calls" PlanTrip -> Schemas :  "validates with"
TripService -> TripAgent :  "invokes" TripService -> APIService :  "calls" TripService -> Helpers :  "uses" TripAgent -> Config : "reads" APIService -> Config :  "reads"
APIService -> Groq :  "HTTPS" APIService -> Nominatim :  "HTTPS" APIService -> OpenMeteo :  "HTTPS" APIService -> Overpass : "HTTPS" APIService -> ORS : "HTTPS" Tools -> APIService :  "wraps"
@enduml
```

Listing 3.1: Component Architecture - PlantUML Code

api.js

- HTTP client wrapper
- API endpoint calls
- Error handling
- Request/response transformation

3.2.2 Backend Components

main.py

```python
app = FastAPI(
    title="AI Trip Planner API",
    description="AI-powered travel itinerary generation",
    version="1.0.0",
    docs_url="/docs"
)

app.add_middleware(CORSMiddleware, ...)
app.include_router(trip_router)

@app.exception_handler(Exception)
async def global_exception_handler(request, exc):
    logger.error(f"Unhandled exception: {exc}")
    return JSONResponse(
        status_code=500,
        content={"status": "error", "message": "Internal server error"}
    )

@app.on_event("startup")
async def startup_event():
    logger.info("Application starting...")
    # Validate configuration
    # Check API keys
```

Listing 3.2: FastAPI Application Initialization

routes/trip_planner.py

```python
@router.post("/api/plan-trip", response_model=TripResponse)
async def plan_trip(request: TripRequest):
    """Generate personalized trip itinerary"""
    # Validate Groq API key
    if not settings.groq_api_key:
        raise HTTPException(
            status_code=500,
            detail="Groq API key not configured"
        )

    # Call service layer
    try:
        response = await trip_service.plan_trip(request)
```

```python
        return response
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    except Exception as e:
        logger.error(f"Trip planning failed: {e}")
        raise HTTPException(status_code=500, detail="Internal error"
            )

@router.get("/api/health")
async def health_check():
    """Health check endpoint"""
    return {"status": "healthy", "service": "AI Trip Planner"}
```

Listing 3.3: API Route Definitions

services/trip_service.py

```python
class TripService:
    async def plan_trip(self, request: TripRequest) -> TripResponse:
        """Main orchestration method"""
        # Calculate trip metrics
        duration = calculate_duration(request.start_date, request.
            end_date)
        daily_budget = request.budget / duration

        # Get destination coordinates
        coords = await api_service.get_coordinates(request.
            destination)

        # Parallel API calls
        weather, pois = await asyncio.gather(
            api_service.get_weather_forecast(coords, ...),
            api_service.get_points_of_interest(coords, ...)
        )

        # Generate itinerary using AI agent
        itinerary = await agent.plan_trip({
            "destination": request.destination,
            "coords": coords,
            "weather": weather,
            "pois": pois,
            "budget": daily_budget,
            "interests": request.interests,
            "pace": request.pace
        })

        return self._format_response(itinerary, request)
```

Listing 3.4: Trip Service Orchestration

agents/trip_agent.py

```python
class TripPlanningAgent:
    def __init__(self):
        self.llm = ChatGroq(
            model="llama-3.1-8b-instant",
```

```python
            temperature=0.7,
            groq_api_key=settings.groq_api_key
        )
        self.tools = trip_planning_tools
        self.agent_executor = create_react_agent(
            model=self.llm,
            tools=self.tools,
            prompt=system_prompt
        )

    def plan_trip(self, trip_request: Dict) -> Dict:
        """Execute agent to generate itinerary"""
        user_query = self._construct_query(trip_request)

        result = self.agent_executor.invoke({
            "input": user_query
        })

        return self._parse_response(result["output"])
```

Listing 3.5: LangChain Agent Implementation

3.3 Data Flow Architecture

The complete request/response flow is illustrated in Figure ??.

[Data Flow Diagram Placeholder]

To generate: Save PlantUML code below to data-flow.puml
then run: java -jar plantuml.jar data-flow.puml

Figure 3.2: Data Flow Diagram

```
@startuml data-flow-diagram
' Data Flow Diagram - Shows complete request/response flow

!theme plain

skinparam activity {
    BackgroundColor LightBlue
    BorderColor Black
    FontSize 10
}

title AI Trip Planner - Data Flow

|User Browser|
start
:User fills trip planning form;
:Validate input client-side;
:Serialize to JSON;

|Frontend API Client|
:Send POST request to /api/plan-trip;
```

```
|Backend FastAPI|
:Receive HTTP request;
:Parse JSON body;
:Validate with Pydantic (TripRequest schema);

if (Validation passed?) then (yes)
    |Backend Routes|
    :Check Groq API key exists;

    if (API key exists?) then (yes)
        |Trip Service|
        :Calculate trip metrics;

        |API Service - Geocoding|
        :Call Nominatim API for destination;
        :Parse coordinates (lat, lon);

        if (Location found?) then (yes)
            |API Service - Parallel Calls|
            fork
                :Get weather forecast (Open-Meteo);
            fork again
                :Get points of interest (Overpass API);
            end fork

            :Aggregate data;

            |Trip Service|
            :Format weather data;
            :Format POI data;

            |AI Agent|
            :Initialize LangChain agent;
            :Execute ReAct pattern;
            :Parse agent final answer (JSON itinerary);

            |Backend Routes|
            :Return HTTP 200 with JSON body;

            |User Browser|
            :Display trip overview;
            :Render daily itineraries;
            :Show recommendations;
            stop
        else (no)
            :Return HTTP 400 Validation Error;
            stop
        endif
    else (no)
        :Return HTTP 500 Configuration Error;
        stop
    endif
else (no)
    :Return HTTP 422 Validation Error;
    stop
endif

```

```
@enduml
```

Listing 3.6: Data Flow Diagram - PlantUML Code

3.4 Deployment Architecture

Figure ?? shows the production deployment topology.

[Deployment Architecture Diagram Placeholder]

To generate: Save PlantUML code below to deployment-architecture.puml then run: java -jar plantuml.jar deployment-architecture.puml

Figure 3.3: Deployment Architecture Diagram

```
@startuml deployment-architecture-diagram
' Deployment Architecture Diagram

!define NODE node

skinparam node {
    BackgroundColor LightBlue
    BorderColor Black
    FontSize 10
}

skinparam cloud {
    BackgroundColor LightYellow
    BorderColor Black
}

skinparam database {
    BackgroundColor LightGreen
    BorderColor Black
}

title AI Trip Planner - Deployment Architecture

cloud "Internet" as Internet

node "Client Machine" {
    [Web Browser] as Browser
}

cloud "Cloud Platform (AWS/Azure/GCP)" {
    node "Load Balancer" {
        [Application Load Balancer] as ALB
    }

    node "Application Server 1" {
        component [Docker Container 1] as Container1 {
            [FastAPI App] as App1
        }
    }

```

```
    node "Application Server 2" {
        component [Docker Container 2] as Container2 {
            [FastAPI App] as App2
        }
    }

    database "Environment Variables" as EnvVars {
        [GROQ_API_KEY]
        [OPENROUTE_API_KEY]
    }

    node "Monitoring" as Monitor {
        [Logs]
        [Metrics]
        [Alerts]
    }
}

cloud "External Services" {
    node "Groq API" as GroqCloud
    node "OpenStreetMap" as OSMCloud
    node "Weather Services" as WeatherCloud
}

Internet -- Browser
Browser --> ALB : HTTPS
ALB --> Container1 : HTTP
ALB --> Container2 : HTTP

Container1 ..> EnvVars : reads
Container2 ..> EnvVars : reads

App1 --> GroqCloud : HTTPS
App1 --> OSMCloud : HTTPS
App1 --> WeatherCloud : HTTPS

App2 --> GroqCloud : HTTPS
App2 --> OSMCloud : HTTPS
App2 --> WeatherCloud : HTTPS

Container1 --> Monitor : Logs & Metrics
Container2 --> Monitor : Logs & Metrics

@enduml
```

Listing 3.7: Deployment Architecture - PlantUML Code

Chapter 4

Data Design

4.1 Data Models Overview

The system uses Pydantic models for data validation and serialization. Figure **??** illustrates the relationships between data models.

[Data Models Diagram Placeholder]

To generate: Save PlantUML code below to data-models.puml
then run: java -jar plantuml.jar data-models.puml

Figure 4.1: Data Models Diagram

```
@startuml data-models
' Data Models Class Diagram

!theme plain

skinparam class {
    BackgroundColor LightBlue
    BorderColor Black
    ArrowColor Black
}

title AI Trip Planner - Data Models

class TripRequest {
    +destination: str
    +start_date: str
    +end_date: str
    +budget: float
    +travelers: int
    +interests: List[str]
    +pace: str
    --validators--
    +validate_start_date()
    +validate_end_date()
    +validate_budget()
}

class TripResponse {
```

```
    +overview: TripOverview
    +itinerary: List[DayItinerary]
    +smart_additions: SmartAdditions
}

class TripOverview {
    +destination: str
    +duration: int
    +start_date: str
    +end_date: str
    +total_budget: float
    +daily_budget: float
    +travelers: int
    +weather_summary: str
    +coordinates: Dict[str, float]
}

class DayItinerary {
    +day: int
    +date: str
    +morning: Activity
    +afternoon: Activity
    +evening: Activity
    +weather: str
    +travel_time: str
    +total_cost: float
    +notes: Optional[str]
}

class Activity {
    +name: str
    +description: str
    +duration: str
    +estimated_cost: float
    +location: Optional[str]
    +time_of_day: str
}

class SmartAdditions {
    +packing_list: List[str]
    +weather_tips: List[str]
    +alternate_activities: List[str]
    +local_tips: List[str]
}

TripResponse "1" *-- "1" TripOverview
TripResponse "1" *-- "*" DayItinerary
TripResponse "1" *-- "1" SmartAdditions
DayItinerary "1" *-- "3" Activity : morning, afternoon, evening

@enduml
```

Listing 4.1: Data Models Diagram - PlantUML Code

4.2 Request Schemas

4.2.1 TripRequest Model

```python
class TripRequest(BaseModel):
    destination: str          # City or location name (min 2 chars)
    start_date: str          # YYYY-MM-DD format
    end_date: str            # YYYY-MM-DD format
    budget: float            # Total budget in USD (>= 100)
    travelers: int           # Number of travelers (1-20)
    interests: List[str]     # e.g., ["culture", "food", "museums"]
    pace: str                # "relaxed", "balanced", or "packed"

    @validator('start_date')
    def validate_start_date(cls, v):
        date = datetime.strptime(v, '%Y-%m-%d').date()
        if date < datetime.now().date():
            raise ValueError('Start date must be in the future')
        return v

    @validator('end_date')
    def validate_end_date(cls, v, values):
        start_date = values.get('start_date')
        if start_date:
            start = datetime.strptime(start_date, '%Y-%m-%d').date()
            end = datetime.strptime(v, '%Y-%m-%d').date()
            if end <= start:
                raise ValueError('End date must be after start date'
                    )
        return v
```

Listing 4.2: TripRequest Pydantic Model

4.2.2 Example Request

Listing 4.3: Example Trip Request JSON

```json
{
  "destination": "Paris, France",
  "start_date": "2026-06-15",
  "end_date": "2026-06-18",
  "budget": 1500.0,
  "travelers": 2,
  "interests": ["culture", "food", "museums", "architecture"],
  "pace": "balanced"
}
```

4.3 Response Schemas

4.3.1 TripResponse Model

```python
class TripResponse(BaseModel):
    overview: TripOverview                  # Trip summary
    itinerary: List[DayItinerary]          # Daily plans
    smart_additions: SmartAdditions        # Recommendations
```

```python

class TripOverview(BaseModel):
    destination: str
    duration: int
    start_date: str
    end_date: str
    total_budget: float
    daily_budget: float
    travelers: int
    weather_summary: str
    coordinates: Dict[str, float]

class DayItinerary(BaseModel):
    day: int
    date: str
    morning: Activity
    afternoon: Activity
    evening: Activity
    weather: str
    travel_time: str
    total_cost: float
    notes: Optional[str]

class Activity(BaseModel):
    name: str
    description: str
    duration: str
    estimated_cost: float
    location: Optional[str]
    time_of_day: str
```

Listing 4.4: TripResponse Pydantic Model

Chapter 5

Interface Design

5.1 User Interface Design

5.1.1 Design Principles

- **Simplicity:** Clean, uncluttered interface
- **Responsiveness:** Mobile-first design, works on all screen sizes
- **Feedback:** Real-time validation and loading states
- **Accessibility:** WCAG 2.1 Level AA compliance
- **Consistency:** Uniform styling and interaction patterns

5.1.2 UI Wireframe

Figure **??** shows the complete user interface layout.

[UI Wireframe Diagram Placeholder]

To generate: Save PlantUML code below to ui-wireframe.puml
then run: java -jar plantuml.jar ui-wireframe.puml

Figure 5.1: User Interface Wireframe

```
@startuml ui-wireframe
' UI Wireframe for Trip Planner Interface

!theme plain

skinparam rectangle {
    BackgroundColor<<header>> LightBlue
    BackgroundColor<<form>> LightGreen
    BackgroundColor<<loading>> LightYellow
    BackgroundColor<<results>> LightCyan
    BackgroundColor<<error>> LightCoral
    BackgroundColor<<footer>> LightGray
    BorderColor Black
}
```

```

title AI Trip Planner - User Interface Wireframe

rectangle "Browser Window" {
    rectangle "Header Section" <<header>> {
        rectangle "AI Trip Planner" as Title
        rectangle "AI-Powered Travel Planning" as Subtitle
    }

    rectangle "Main Content Area" {
        rectangle "Input Section" <<form>> {
            rectangle "Trip Planning Form" {
                rectangle "Destination Input"
                rectangle "Start Date Picker"
                rectangle "End Date Picker"
                rectangle "Budget Input"
                rectangle "Travelers Count"
                rectangle "Interests Checkboxes"
                rectangle "Travel Pace Selector"

                rectangle "Submit Button" <<header>> {
                    rectangle "Plan My Trip"
                }
            }
        }

        rectangle "Loading Section" <<loading>> {
            rectangle "Loading Animation"
            rectangle "Progress Bar"
            rectangle "Status Messages"
        }

        rectangle "Results Section" <<results>> {
            rectangle "Trip Overview Card"
            rectangle "Daily Itinerary Container"
            rectangle "Smart Recommendations"
            rectangle "Action Buttons"
        }

        rectangle "Error Section" <<error>> {
            rectangle "Error Banner"
            rectangle "Error Message"
            rectangle "Retry Button"
        }
    }

    rectangle "Footer Section" <<footer>> {
        rectangle "Copyright"
        rectangle "Links"
    }
}

@enduml
```

Listing 5.1: UI Wireframe - PlantUML Code

5.2 API Interface Design

5.2.1 REST API Endpoints

Endpoint	Method	Purpose	Auth
`/api/plan-trip`	POST	Generate itinerary	None
`/api/health`	GET	Health check	None
`/api/sample-request`	GET	Example request	None
`/docs`	GET	Swagger UI	None
`/redoc`	GET	ReDoc UI	None

Table 5.1: API Endpoints

Chapter 6

Security Design

6.1 Security Architecture

Figure ?? illustrates the multi-layered security architecture.

[Security Architecture Diagram Placeholder]

To generate: Save PlantUML code below to security-architecture.puml then run: java -jar plantuml.jar security-architecture.puml

Figure 6.1: Security Architecture Diagram

```
@startuml security-architecture-diagram
' Security Architecture Diagram

!theme plain

skinparam rectangle {
    BackgroundColor<<security>> LightYellow
    BackgroundColor<<data>> LightBlue
    BackgroundColor<<network>> LightGreen
    BorderColor Black
}

title AI Trip Planner - Security Architecture

rectangle "Client Security Layer" <<security>> {
    rectangle "Browser Security" {
        component [HTTPS/TLS]
        component [Content Security Policy]
        component [Input Validation]
        component [XSS Prevention]
    }
}

rectangle "Network Security Layer" <<network>> {
    rectangle "Transport Security" {
        component [TLS 1.2+]
        component [SSL Certificates]
        component [HSTS Headers]
    }
```

```

    rectangle "API Security" {
        component [CORS Policy]
        component [Rate Limiting]
        component [IP Filtering]
    }
}

rectangle "Application Security Layer" <<security>> {
    rectangle "Input Security" {
        component [Pydantic Validation]
        component [Type Checking]
        component [String Sanitization]
    }

    rectangle "Output Security" {
        component [JSON Encoding]
        component [HTML Escaping]
        component [Error Masking]
    }
}

rectangle "Data Security Layer" <<data>> {
    rectangle "Secrets Management" {
        component [Environment Variables]
        component [API Key Storage]
        component [Key Masking in Logs]
    }
}

[Browser Security] --> [Transport Security]
[Transport Security] --> [API Security]
[API Security] --> [Input Security]
[Input Security] --> [Output Security]
[Output Security] --> [Secrets Management]

@enduml
```

Listing 6.1: Security Architecture - PlantUML Code

6.2 Authentication and Authorization

6.2.1 Current State

- No user authentication (public API)
- No authorization required
- Rate limiting by IP address

6.2.2 Future Enhancements

- JWT-based authentication
- API key authentication for programmatic access

- Role-based access control (RBAC)
- OAuth2 integration

6.3 Data Security

6.3.1 Input Validation

```python
@validator('destination')
def validate_destination(cls, v):
    if len(v) < 2:
        raise ValueError("Destination too short")
    if not v.strip():
        raise ValueError("Destination required")
    # Sanitize HTML/script tags
    return v.strip()

@validator('budget')
def validate_budget(cls, v):
    if v < 100:
        raise ValueError("Budget must be at least $100")
    if v > 1000000:
        raise ValueError("Budget exceeds maximum")
    return v
```

Listing 6.2: Pydantic Validators

Summary

This design document provides a comprehensive technical blueprint for the AI Trip Planner Agent system. All architectural diagrams are rendered directly from PlantUML source code, ensuring accuracy and maintainability.

Key Takeaways

- Three-tier architecture with clear separation of concerns
- ReAct agent pattern for intelligent trip planning
- FastAPI backend with async/await for high performance
- Comprehensive security layers from client to data
- Containerized deployment ready for cloud platforms
- Rendered diagrams directly in PDF for easy viewing

Next Steps

1. Review and approve architecture design
2. Begin implementation following component specifications
3. Set up CI/CD pipeline for automated testing and deployment
4. Implement monitoring and logging infrastructure
5. Conduct security audit before production deployment

Appendix A

PlantUML Diagram Generation Guide

This document includes PlantUML code for all system diagrams. Diagrams are provided as code listings throughout the document. To generate visual diagrams, follow these steps.

A.1 Prerequisites

1. **Java Runtime Environment (JRE):** Required for PlantUML

```
# Verify Java installation
java -version  # Should show Java 8 or higher
```

2. **PlantUML Installation:**

   - Download `plantuml.jar` from https://plantuml.com/download
   - Save to a convenient location (e.g., `C:\tools` or `/usr/local/bin`)

3. **Graphviz (Recommended):**

```
# Windows (with Chocolatey)
choco install graphviz

# macOS
brew install graphviz

# Linux
sudo apt-get install graphviz
```

A.2 Document Compilation

A.2.1 Standard Compilation (Without Diagrams)

```
# Compile document (PlantUML code shown as listings)
pdflatex DESIGN-DOCUMENT-WITH-DIAGRAMS.tex

# Run twice for proper TOC and cross-references
pdflatex DESIGN-DOCUMENT-WITH-DIAGRAMS.tex
```

A.2.2 Alternative: Using latexmk

```
latexmk -pdf DESIGN-DOCUMENT-WITH-DIAGRAMS.tex
```

A.3 Generating Diagram Images

A.3.1 Step 1: Extract PlantUML Code

The document contains PlantUML code listings for 8 diagrams:

1. System Context Diagram (system-context-diagram.puml)
2. High-Level Architecture (high-level-architecture.puml)
3. Component Architecture (component-architecture.puml)
4. Data Flow Diagram (data-flow.puml)
5. Deployment Architecture (deployment-architecture.puml)
6. Data Models (data-models.puml)
7. UI Wireframe (ui-wireframe.puml)
8. Security Architecture (security-architecture.puml)

Copy each PlantUML code block to a separate `.puml` file with the name shown above.

A.3.2 Step 2: Generate PNG Images

```
# Generate all diagrams at once
java -jar plantuml.jar *.puml

# Or generate individually
java -jar plantuml.jar system-context-diagram.puml
java -jar plantuml.jar high-level-architecture.puml
# ... etc
```

This creates PNG images for each diagram.

A.3.3 Step 3: View Diagrams

- Open the generated PNG files in any image viewer
- Or incorporate them into presentations, documentation, etc.
- Or replace the placeholder boxes in the LaTeX document with `\includegraphics` commands

A.4 Troubleshooting

A.4.1 Error: Java not found

Solution: Install Java JRE:

- Windows: Download from https://www.java.com/download/
- macOS: `brew install openjdk`
- Linux: `sudo apt install default-jre`

A.4.2 Error: Cannot find plantuml.jar

Solution: Specify full path to PlantUML:

```
# Windows
java -jar "C:\tools\plantuml.jar" *.puml

# Linux/Mac
java -jar /usr/local/bin/plantuml.jar *.puml
```

A.4.3 Diagram Layout Issues

Solution: Install Graphviz for better diagram layouts:

- PlantUML uses Graphviz for complex diagram layouts
- Without Graphviz, some diagrams may have poor layouts
- Install from https://graphviz.org/download/

A.5 Advanced: Integrating Diagrams into LaTeX

If you want the generated diagrams embedded in the PDF:

A.5.1 Option 1: Manual Replacement

1. Generate all PNG images as described above
2. Copy PNG files to your LaTeX project directory
3. Replace placeholder boxes with:

   ```
   \includegraphics[width=0.9\textwidth]{diagram-name.png}
   ```

4. Recompile LaTeX document

A.5.2 Option 2: Automated with Makefile

Create a `Makefile`:

```
all: diagrams document

diagrams:
        java -jar plantuml.jar *.puml

document:
        pdflatex DESIGN-DOCUMENT-WITH-DIAGRAMS.tex
        pdflatex DESIGN-DOCUMENT-WITH-DIAGRAMS.tex

clean:
        rm -f *.png *.aux *.log *.toc *.out
```

Then run: `make`

A.6 Tips for Best Results

- **High Resolution:** Use `-Sdpi=300` flag for print quality

  ```
  java -jar plantuml.jar -Sdpi=300 *.puml
  ```

- **Different Formats:** Generate SVG for scalability

  ```
  java -jar plantuml.jar -tsvg *.puml
  ```

- **Batch Processing:** Process all diagrams in one command

  ```
  java -jar plantuml.jar **/*.puml
  ```

Document Approval

Role	Name	Signature	Date
Lead Architect			
Backend Lead			
Frontend Lead			
DevOps Lead			
QA Lead			
Technical Director			
Product Manager			

— End of Design Document —

This document is subject to change. All updates should be reviewed
and approved by the technical leads before implementation.

Document Classification: Confidential - Internal Use Only
Version: 1.0
Date: February 25, 2026
Next Review: August 25, 2026

Compilation Info:
Standard compilation: pdflatex DESIGN-DOCUMENT-WITH-DIAGRAMS.tex
Diagrams: Provided as PlantUML code listings
To generate diagram images: See Appendix A